

台灣自編大學教科書

@prof

機器人學導論：機構、感知、控制與具身人工智慧

第 2 章

如何學習機器人學

——世界名校課程巡禮

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】一張向上盤旋的螺旋樓梯照片，每一階梯上放置一台由簡到繁的機器人——震動蟲、循跡小車、自平衡車、機械手臂、四足機器狗——象徵螺旋式學習：同一個問題，每繞一圈，工具更強、理解更深。

叫獸開講 「先學數學，還是先讓機器人動起來？」

開學第二週，叫獸的研究室來了兩位一年級學生。他們是高中同班同學，一起考進智慧機器人學程，卻在第一堂課後吵了起來，決定找老師評理。

阿哲說：「我覺得應該先把數學打好。線性代數、微積分、機率論，這些不學會，後面的運動學、動力學根本看不懂。我打算大一大二先把數學修完，大三再開始碰機器人。」他把一疊列印好的課表推到桌上，上面用螢光筆標了滿滿的必修數學。

小妤搖頭：「那太慢了。我國中就在社團做過循跡小車，也玩過 Arduino。我覺得應該先讓機器人動起來，遇到問題再回頭補數學。你先學三年數學，中間一台機器人都沒做出來，早就忘光了。」她從背包裡拿出一台自己焊的小車，輪子還沾著賽道的黑膠帶屑。

叫獸聽完，沒有站在任何一邊，反而問了一個問題：「你們兩個都說得有道理。但我想先問——阿哲，你知不知道，六軸手臂為什麼需要用到矩陣？小妤，你知不知道，你那台循跡小車為什麼在轉彎太急的時候會衝出賽道？」

兩個人都愣住了。阿哲確實不知道矩陣在機器人裡具體要拿來做什麼，他只知道「線性代數很重要」；小妤知道車子會衝出去，但她的解法是「把速度調慢一點」，至於為什麼調慢就好、該調多慢，她說不上來。

叫獸笑了：「這就是問題所在。阿哲的路線，是把工具磨得很利，卻不知道要砍哪棵樹；小妤的路線，是拿著斧頭一直砍，卻不知道換一把利刃可以省一半力氣。機器人學最迷人也最難的地方，就在於它是一門『雙向』的學問——理論要落到會動的機器上才有意義，動手做遇到瓶頸時，才會知道該去學哪個數學。」

他站起來，在白板上畫了一個往上盤旋的螺旋：「正確的學法不是直線，是螺旋。做一台循跡小車，遇到轉彎衝出去，回頭學 PID 控制；學會 PID，再做一台會平衡的兩輪車，發現需要角度感測，回頭學 IMU 與濾波；學會濾波，再做定位建圖，發現需要機率論.....每繞一圈，你的手更巧、你的數學更深，而且你永遠知道，這個公式是為了解決哪一個具體的痛苦而存在的。」

小妤問：「那我們該從哪裡開始繞第一圈？」叫獸把那台沾著膠帶屑的小車拿起來，說：「你已經開始了。而阿哲——你要做的，是把你手上那些數學，一個一個接到真實的機器上。這一章，我就帶你們看看，全世界最頂尖的大學是怎麼安排這條螺旋的。」

本章為什麼重要

第 1 章回答了「機器人是什麼」，本章回答「機器人學該怎麼學」。這不是可有可無的方法論閒談：機器人學橫跨機械、電機、電子、控制、資訊與人工智慧六大領域，是典型的「寬而深」學科。沒有學習策略的人，很容易掉進兩個陷阱——一是被數學公式淹沒，讀了兩年還沒讓任何東西動起來；二是一直停留在接線與複製程式碼，遇到瓶頸卻不知道該補哪塊理論。本章提供三樣東西：一張知識地圖（知道全貌在哪）、一份數學與工具清單（知道要準備什麼）、一套經過世界名校驗證的學習路徑（知道別人怎麼走成功的）。讀完之後，你手上會有一份屬於自己的四年（或三年、或一學期）學習計畫。

學習目標

- 能畫出機器人學的知識階層圖，並說明各層之間的依賴關係。
- 能說明「螺旋式學習」的意義，並為自己規劃至少三圈的學習循環。
- 能列舉機器人學所需的四大數學工具，並各舉一個在本書中的具體應用章節。
- 能說明向量、矩陣、三角函數與微積分在機器人學中分別解決什麼問題。
- 能區分 Python、C/C++ 與圖形化程式在機器人開發中的分工與適用場合。
- 能說明模擬器、數位分身與實體機器人三者的關係與各自的優缺點。
- 能比較 Stanford、MIT、CMU、柏林工業大學與 ETH Zürich 五所名校機器人課程的側重差異。
- 能說明問題導向學習（PBL）與專題導向學習的差異，並設計一個屬於自己的入門專題。
- 能依自己的身分（大學生 / 高中生 / 國中小社團）選定本書的閱讀路線與進度。
- 能完成本書所需的軟硬體環境建置清單，並評估自己的裝備缺口。

先備知識

僅需第 1 章的內容。本章不進行任何推導，所有數學都只談「它能解決什麼問題」，不談「怎麼算」。若你在閱讀 2.3、2.4 節時感到陌生，那完全正常——這正是本章的目的：讓你知道自己還缺什麼，而不是現在就學會它。

核心名詞中英對照

表 2-1 本章核心名詞

中文	英文	說明
螺旋式學習	Spiral Learning	同一主題在不同深度反覆出現、逐圈加深的學習法
問題導向學習	Problem-Based Learning, PBL	以真實問題驅動知識學習的教學法
專題導向學習	Project-Based Learning	以完成具體作品為目標的教學法
先備知識	Prerequisite	學習某主題前必須具備的基礎知識
模擬器	Simulator	以軟體重現機器人與環境物理行為的工具
數位分身	Digital Twin	與實體設備即時對應、同步更新的虛擬模型
模擬到實機落差	Sim-to-Real Gap	模擬環境與真實世界之間的行為差異
整合開發環境	Integrated Development Environment, IDE	整合編輯、編譯與除錯的程式開發軟體
版本控制	Version Control	追蹤程式碼變更歷史的管理系統，如 Git
開源軟體	Open Source Software	原始碼公開、可自由使用與修改的軟體
機器人作業系統	Robot Operating System, ROS	機器人軟體開發的中介框架與生態系
總整專題	Capstone Project	整合全部所學、於學程末期完成的綜合性專題

2.1 機器人學的知識階層

2.1.1 六層金字塔

機器人學的知識可以整理成一座六層的金字塔，由下而上分別是：物理與數學基礎、機構與硬體、幾何與運動學、力學與控制、感知與規劃、學習與智慧。每一層都以下一層為地基，但地基不必全部完工才能往上蓋——這正是與傳統理工科系「先修課全部修完才能修進階課」的線性思維最大的不同。

最底層是物理與數學基礎：牛頓力學、線性代數、微積分、機率論。第二層是機構與硬體：連桿、關節、馬達、感測器、電子電路（第 3~7 章）。第三層是幾何與運動學：座標轉換、正逆運動學、雅可比矩陣（第 8~12 章）。第四層是力學與控制：動力學、軌跡生成、PID 與最佳控制（第 13~21 章）。第五層是感知與規劃：視覺、狀態估測、SLAM、路徑規劃（第 22~28

章)。最上層是學習與智慧：機器學習、強化學習、大型模型與具身智慧(第 29~33 章)。

圖 2-1 機器人學知識階層金字塔

建議配圖：六層金字塔，由下而上為物理與數學、機構與硬體、幾何與運動學、力學與控制、感知與規劃、學習與智慧。右側標註對應章節；左側以一條向上盤旋的箭頭穿過各層，代表螺旋式學習不必逐層完工。

2.1.2 依賴關係：哪些是硬性、哪些是軟性

層與層之間的依賴有強有弱，分辨清楚可以省下大量時間。硬性依賴是「不懂前者就真的看不懂後者」：不懂矩陣乘法，就無法理解齊次變換；不懂微分方程，就無法理解 PD 控制的解；不懂機率的條件與邊際分佈，就無法理解 Bayes 濾波。軟性依賴則是「先跳過也能懂個大概」：不懂拉格朗日力學，仍然可以會用 PID 讓手臂動起來；不懂 SLAM 的數學，仍然可以操作 ROS 2 的 SLAM 套件建出地圖。

對初學者而言，最有效率的策略是：硬性依賴老老實實補，軟性依賴大膽先跳。本書每章開頭的「先備知識」欄位，就是為此設計的——它明確告訴你，這一章真正需要什麼、什麼可以之後再說。

2.1.3 三種常見的學習誤區

誤區一是「數學無底洞」：認為要先把所有數學讀完才能開始，結果永遠沒有開始。事實上機器人學用到的數學範圍雖廣，但深度有限——線性代數用到矩陣運算與奇異值分解、微積分用到偏導數與微分方程、機率用到 Bayes 定理與高斯分佈，多數在大一大二的課程範圍內。

誤區二是「接線就是機器人」：能把馬達接起來讓車子跑，就以為學會了機器人。硬體是入場券不是畢業證書；真正的門檻在於，當車子跑偏、手臂抖動、地圖漂移時，你能不能診斷出是機構、控制還是感知出了問題。

誤區三是「只讀論文不動手」：能講出十種 SLAM 演算法的差異，卻沒有讓任何一台機器人真的跑起來。機器人學是一門會被現實無情打臉的學科——摩擦力、雜訊、電磁干擾、電池電壓下降，這些課本上一行帶過的東西，會在實驗室裡花掉你整個週末。動手，是唯一能學到這些的方法。

2.2 由觀察、組裝、程式到數學模型

2.2.1 四階段學習法

把叫獸白板上的螺旋展開，每一圈都包含四個階段：觀察、組裝、程式、數學模型。這四階段的順序不是隨意的——它符合人類認知由具體到抽象的自然歷程。

觀察階段：看真實的機器人如何運作。看掃地機器人怎麼繞開椅腳、看工廠手臂怎麼取放工件、看影片裡的機器狗怎麼在雪地上恢復平衡。這階段的產出是「問題」——為什麼它要這樣動？如果換成我，我會怎麼做？

組裝階段：親手把一台會動的東西做出來。從附錄 A 的震動機器人到附錄 F 的循跡小車，重點不在做得多精緻，而在於你必須面對真實的物理：螺絲會鬆、電線會斷、電池會沒電、馬達左右轉速不一樣。

程式階段：讓它按照你的意志動作。從圖形化積木到 Python，從「前進三秒」到「偵測到黑線就修正」。這階段你會第一次體驗到「感知—思考—行動」不是抽象概念，而是三段真實的程式碼。

數學模型階段：用公式描述並預測它的行為。到這裡你才會問：為什麼 P 增益調太大會震盪？該調多少才不震盪？——而這正是第 16 章要回答的問題。因為你已經親眼看過震盪，這個公式對你來說就不再是死背，而是解藥。

表 2-2 四階段學習法的內容與產出

階段	做什麼	典型活動	產出	常見卡關
觀察	看真實機器人如何運作	工廠參訪、影片分析、拆解舊設備	一份「為什麼」清單	只看熱鬧不看門道
組裝	親手做出會動的東西	附錄 A~E 手作單元	一台會動的機器	只求能動、不求理解
程式	讓它按你的意志動作	圖形化程式、Python 控制	可重複執行的行為	抄程式碼而不改參數
數學模型	用公式描述與預測行為	推導、模擬、參數設計	可預測、可設計的系统	推導完不回頭驗證實機

2.2.2 螺旋的三圈範例

第一圈（入門）：組裝循跡小車 → 用圖形化程式讓它跑 → 發現轉彎衝出去 → 學會「偏差越大、修正越大」的比例概念。此圈的數學深度：小學高年級的比例。

第二圈（進階）：改用 Python 控制 → 加入速度限制與微分項 → 發現原地抖動 → 學會 PD 控制與阻尼的概念（第 16 章）。此圈的數學深度：高中的微分直覺。

第三圈（專業）：改用兩輪自平衡車 → 加入 IMU → 發現感測器雜訊讓角度亂跳 → 學會互補濾波與 Kalman 濾波（第 24 章）→ 進一步推導系統的動力學方程與 LQR 控制器（第 18 章）。此圈的數學深度：大學的線性代數、微分方程與機率論。

同一個主題（讓車子穩定地走），在三圈之中被反覆處理，但每一圈的工具都更強、理解都更深。這就是螺旋式學習的威力——也是本書為什麼要把附錄 A~L 的社團實作與正文章節用「螺旋銜接」欄綁在一起的原因。

◎ 叫獸的建議

如果你是大一新生，請在第一學期就做至少一台會動的機器人，不論多簡陋。原因不是技術上的，是心理上的：親手讓某個東西動起來的成就感，會支撐你度過後面兩年枯燥的數學課。而當你在解那些矩陣時，腦中會有一台具體的機器在轉動——這會讓一切變得完全不同。

2.3 機器人學所需的數學基礎

2.3.1 四大工具與它們的任務

機器人學的數學不是無邊無際的。它主要用到四樣工具，而且每一樣都有明確的任務。

線性代數負責「空間」：機器人在三維空間中的位置與姿態、座標系之間的轉換、多個關節的速度如何合成末端速度——這些全部靠向量與矩陣描述。它是機器人學最核心、使用最頻繁的數學，出現在第 8~12 章的每一頁。

微積分負責「變化」：速度是位置的導數、加速度是速度的導數、能量的變化率決定運動方程。控制系統的穩定性分析、軌跡的平滑度、最佳化的梯度下降，全都建立在微分之上。它主要出現在第 13~21 章。

機率論負責「不確定性」：感測器有雜訊、馬達有誤差、環境會變化。當機器人問「我在哪裡」時，正確的答案不是一個點，而是一個機率分佈。這是第 24、25 章的靈魂。

最佳化負責「選擇」：逆運動學有無限多組解，選哪一組？路徑有無限多條，走哪一條？控制器參數有無限多種，用哪一組？最佳化就是把「選擇」變成「求極值」的數學。它貫穿第 10 章的 IK、第 18 章的最佳控制與第 26 章的軌跡最佳化。

表 2-3 四大數學工具與本書對應章節

數學工具	負責的問題	關鍵概念	本書主要章節
線性代數	空間與姿態	向量、矩陣、行列式、秩、奇異值分解、特徵值	第 8 ~ 12、19 章
微積分	變化與運動	導數、偏導數、微分方程、梯度、泰勒展開	第 13 ~ 21 章
機率論	不確定性	條件機率、Bayes 定理、高斯分佈、期望值	第 24 ~ 25、31 章
最佳化	選擇與設計	目標函數、約束、梯度下降、Gauss-Newton	第 10、18、26 章

2.3.2 要學到多深

學生最常問的問題是：「這些數學要學到什麼程度？」誠實的答案是：以本書的範圍而言，扎實的大一大二數學就足夠。具體而言，線性代數需要熟悉矩陣乘法、反矩陣、行列式、正交矩陣與特徵值，並對奇異值分解有概念性理解；微積分需要熟悉偏導數與鏈鎖律，並能解簡單的二階常微分方程；機率論需要熟悉條件機率、Bayes 定理、期望值與變異數，以及多變量高斯分佈的基本性質；最佳化則需要理解「找極值就是找梯度為零的點」，並知道 Gauss-Newton 法的基本想法。

如果你打算往研究所發展，還會用到李群與李代數（描述旋轉的更嚴謹框架）、凸最佳化、隨機過程與資訊理論。但那是第二階段的事——本書會在相關章節的「進階推導」專欄中指出方向，不會讓初學者卡在門口。

2.3.3 補救策略：即時學習

如果你的數學基礎不夠，不必因此停下腳步。最有效的策略是「即時學習」（just-in-time learning）：不預先把整本線性代數讀完，而是在讀到第 8 章需要旋轉矩陣時，才回頭把正交矩陣的性質補起來。這樣做有兩個好處：一是動機明確，你知道這個概念要拿來做什麼；二是記憶牢固，因為它立刻被用在一個具體的問題上。附錄 D 的數學補給站正是為此設計，它只收錄本書真正會用到的部分，每一則都附上「這在第幾章會用到」的標記。

2.4 向量、矩陣、三角函數與微積分

2.4.1 向量：位置與方向的語言

向量是機器人學的第一個詞彙。末端執行器的位置是一個三維向量、關節角度的集合是一個 n 維向量、施加的力與力矩也是向量。你需要熟悉的操作有：加減（位移的疊加）、內積（計算

投影與夾角，也用於功與能量）、外積（計算力矩與法向量，在第 11 章的雅可比推導中大量使用）、範數（計算距離與誤差大小）。

一個具體的例子：機器手臂末端目前在位置 p ，目標在 p^* ，那麼誤差向量就是 $p^* - p$ ，它的範數是距離，它的方向指出該往哪走。第 10 章的逆運動學，本質上就是不斷縮短這個誤差向量的過程。

2.4.2 矩陣：轉換與映射的語言

矩陣在機器人學中主要扮演兩個角色。第一是描述座標轉換：一個 3×3 的旋轉矩陣描述姿態，一個 4×4 的齊次變換矩陣同時描述位置與姿態（第 8 章）。第二是描述線性映射：雅可比矩陣把關節速度映射到末端速度（第 11 章）、慣性矩陣把加速度映射到力矩（第 13 章）。

矩陣的一些性質在機器人學中有極其具體的物理意義，這是本書會反覆強調的重點。例如：旋轉矩陣是正交矩陣，所以它的反矩陣等於轉置——這讓座標的反向轉換變得極其便宜；雅可比矩陣的秩下降代表機器人進入奇異點，某個方向的運動能力瞬間消失（第 12 章）；慣性矩陣是正定矩陣，所以它必定可逆，這保證了正向動力學一定有解（第 14 章）。當你在線性代數課上覺得「特徵值到底有什麼用」時，請記住：第 19 章會用特徵值的正負，判斷一台機器人會不會失控。

2.4.3 三角函數：關節與姿態的橋樑

三角函數是連接「關節轉了幾度」與「末端在哪裡」的橋樑。平面二連桿手臂的正向運動學，就是兩個餘弦與兩個正弦的組合；逆向運動學則需要反三角函數，並且會遇到「一個位置對應兩組角度」的多解問題（手肘朝上或朝下，第 10 章）。程式實作上有一個實務要點：務必使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ，因為前者能正確判斷四個象限——這個小細節，是初學者除錯時最常掉進去的坑之一。

2.4.4 微積分：從運動到控制

微積分在機器人學的第一個用途是描述運動的層級關係：位置微分得到速度，速度微分得到加速度，加速度再微分得到急動度（jerk，第 15 章討論軌跡平滑度時會用到）。第二個用途是建立運動方程：拉格朗日方法用能量對廣義座標的偏導數，直接推導出動力學方程（第 13 章）。第三個用途是分析與設計控制器：PD 控制的閉環系統是一個二階常微分方程，它的解決定了系統是震盪、臨界阻尼還是過阻尼（第 16 章）。第四個用途是最佳化：梯度是函數上升最快的方

向，梯度下降就是沿著反方向走到谷底（第 10、26 章）。

◎ 數學不是門檻，是捷徑

許多學生把數學視為機器人學的門檻，其實它是捷徑。沒有數學，你只能靠試誤法調參數，一組一組試到天亮；有了數學，你可以直接算出「要讓系統在零點五秒內穩定且不超越， K_p 該設多少、 K_d 該設多少」（第 16 章）。前者是碰運氣，後者是工程。

2.5 力學、電子電路與控制基礎

2.5.1 力學：機器人為什麼會動、會抖、會壞

機器人是實體的機器，因此古典力學是它的基本法則。你需要的力學知識包括：牛頓定律（力等於質量乘加速度，這是第 14 章 Newton-Euler 遞迴的出發點）、力矩與轉動慣量（決定馬達選型，第 5 章）、能量與功（拉格朗日方法的基礎，第 13 章）、摩擦與背隙（決定實際精度的隱形殺手）、剛性與共振（決定手臂能跑多快而不抖動，第 27 章）。

一個實務上的例子：學生設計手臂時常常只計算「馬達扭力夠不夠舉起負載」，卻忘了計算「加速時需要多少額外扭力」與「連桿的彈性變形有多大」。前者讓手臂舉不動，後者讓手臂末端在高速運動時抖動不止——兩者都是力學問題。

2.5.2 電子電路：讓訊號與能量正確流動

電子電路在機器人中扮演兩條路線：訊號路線（感測器讀值、通訊、控制指令）與功率路線（馬達驅動、電池供電）。初學者需要掌握的最小知識集合是：歐姆定律與功率計算（估算電池撐多久、線徑要多粗）、數位與類比訊號的差別、PWM 如何控制馬達轉速、常見通訊介面（UART、I²C、SPI）的用途、以及最重要的——共地、接地與電磁干擾的基本觀念。

為什麼最後一項最重要？因為機器人是「大電流馬達」與「微弱訊號感測器」共存的系統。馬達啟動瞬間的電流突波，會讓電源電壓瞬間下降、讓感測器讀值亂跳、甚至讓微控制器重新開機——這正是第 1 章與第 7 章都會提到的「明明接好了電線，為什麼一動就重開機」的元凶。學會分離電源、加裝濾波電容、正確佈置接地，往往比多寫一百行程式更能救活一台機器人。

2.5.3 控制基礎：從開迴路到閉迴路

控制是機器人學的中樞神經。入門階段你需要建立三個直覺。第一，開迴路與閉迴路的差別：前者是「我叫你走三秒」，後者是「我看著你走到定位為止」——後者才能對抗摩擦、負載變化

與外力干擾。第二，回授的三種成分：比例（現在差多少）、微分（正在以多快的速度接近）、積分（過去累積了多少沒補上的誤差），這就是 PID。第三，穩定性的概念：增益太小反應遲鈍，增益太大則會震盪甚至發散——這個「太大会爆炸」的直覺，是第 19 章 Lyapunov 穩定性分析的樸素版本。

好消息是，這三個直覺不需要數學就能先建立，你可以在附錄 F 的循跡小車上親眼看到。壞消息是，要把它們變成可設計、可預測的工程，就必須回到微分方程——這就是第 16 章存在的理由。

2.6 Python、C/C++ 與圖形化程式

2.6.1 三種語言的分工

機器人開發不是只用一種語言。實務上，三種工具各有各的位置，硬要用一種打天下反而事倍功半。

圖形化程式（Scratch、mBlock、micro:bit MakeCode、Blockly）以拖拉積木取代打字，適合國中小社團與完全零基礎的入門者。它的價值不在於能做多複雜的專案，而在於它讓學習者把注意力集中在「邏輯」上，而非「分號忘了打」。附錄 B、F、G、H 的社團單元皆以此為主要工具。

Python 是今日機器人研究與快速原型的主流語言。它語法簡潔、函式庫豐富（NumPy 做矩陣運算、OpenCV 做影像處理、PyTorch 做深度學習），且是 ROS 2 的官方支援語言之一。本書的所有課文程式範例都以 Python 撰寫，理由很簡單：讀者可以把注意力放在演算法本身，而不是記憶體管理。

C/C++ 則是即時控制與嵌入式系統的王者。當你需要在「一毫秒內完成一次控制迴路運算」，或是在只有幾十 KB 記憶體的低控制器上跑程式時，C/C++ 幾乎是唯一選擇。工業機器人控制器、飛控韌體、ROS 2 的核心底層，全都是 C++ 寫的。Arduino 使用的也是 C++ 的簡化框架。

表 2-4 三種程式工具的比較與適用場合

工具	優點	缺點	典型場合	本書使用處
圖形化程式	零門檻、視覺化、不會語法錯誤	難以處理複雜邏輯與大型專案	國中小社團、概念示範	附錄 B、F、G、H

工具	優點	缺點	典型場合	本書使用處
Python	語法簡潔、函式庫豐富、開發快	執行速度較慢、即時性不足	演算法原型、資料分析、AI	全書課文範例
C/C++	執行快、可即時、可跑在微控制器	語法嚴格、開發較慢、易出錯	嵌入式韌體、即時控制迴路	第 7、16 章、附錄 C ~ E

2.6.2 學習順序建議

對零基礎者，建議的順序是：圖形化程式（建立邏輯直覺）→ Python（學會真正的程式設計）→ C/C++（處理即時與嵌入式）。對已有程式基礎的大學生，可直接從 Python 開始，在第 7 章接觸微控制器時再學 Arduino 風格的 C++。

有一個常見的迷思需要澄清：許多學生認為「C++ 比較高級、Python 是玩具」。事實正好相反——現代機器人系統幾乎都是混合架構：底層的即時控制迴路用 C++ 寫、上層的規劃與決策用 Python 寫，兩者透過 ROS 2 的訊息機制溝通。會用對地方，比會用哪一種更重要。

2.6.3 不只是語言：三樣必備的工程習慣

除了語言本身，有三樣工程習慣會決定你走得多遠。第一是版本控制（Git）：機器人專案的程式碼會不斷修改，沒有版本控制，你會在「昨天明明可以跑」的地獄裡浪費無數時間。第二是註解與文件：三個月後的你，就是另一個人；沒有註解的程式碼，等於沒寫。第三是模組化：把感知、決策、控制拆成獨立的函式或節點，才能單獨測試、單獨除錯——這正是 ROS 2 節點架構背後的哲學（第 29 章）。

◎ 除錯的黃金法則

當機器人不動時，請依序檢查：一、有電嗎（電池電壓、電源開關）？二、線接對了嗎（正負、訊號腳位）？三、程式跑到那一行了嗎（加 print 或 LED 指示）？四、感測器讀值合理嗎（印出來看）？五、才是懷疑演算法。初學者最常犯的錯，是第一步就跳到第五步，花三小時檢查演算法，最後發現是電池沒電。

2.7 模擬、數位分身與實體機器人的關係

2.7.1 為什麼要用模擬器

模擬器是用軟體重現機器人與環境的物理行為。它有四個不可取代的價值。第一是零成本試錯：在模擬中撞牆一百次不會壞任何東西，在真實世界撞一次可能就是幾萬元的維修費。第二是

速度：強化學習可能需要百萬次嘗試，真實機器人做不到，模擬器可以用比實時快數十倍的速度跑。第三是可重複性：真實世界的每一次實驗條件都略有不同，模擬器可以完美重現同一個場景，方便比較演算法優劣。第四是安全性：測試一個尚未驗證的控制器，在模擬中失控只是畫面亂跳，在真機上失控可能傷人。

常見的機器人模擬器包括：Gazebo（與 ROS 2 深度整合，本書第 30 章使用）、Webots（教學友善、跨平台）、Isaac Sim（NVIDIA 出品，圖形逼真、適合 AI 訓練）、MuJoCo（物理精確、廣泛用於強化學習研究）、CoppeliaSim（原名 V-REP，教學界常用）。工業界則有 ABB RobotStudio、FANUC ROBOGUIDE 等原廠離線程式軟體。

2.7.2 數位分身：模擬器的產業版本

數位分身（Digital Twin）比一般模擬器更進一步：它與實體設備即時連線、同步更新狀態。工廠裡的實體產線在運轉，電腦裡的虛擬產線同步顯示每一台機器的位置、溫度與稼動率。它的用途包括虛擬調試（在設備到廠前先驗證程式）、預知保養（從數據預測何時該換軸承）與製程最佳化（在虛擬世界試新排程，不影響生產）。第 30 章會完整展開這個主題。

2.7.3 模擬到實機的落差

必須誠實面對的是：模擬永遠不等於現實。這個差距稱為 Sim-to-Real Gap，主要來源有四個。一是接觸與摩擦的物理難以精確建模——這是模擬中最不可靠的部分，也是為什麼在模擬中完美的抓取，到真機上常常滑落。二是感測器雜訊的特性難以完整重現——真實的相機有動態範圍限制、真實的 LiDAR 在玻璃上會失效。三是致動器的非理想性——背隙、摩擦、發熱後的性能衰減。四是環境的不可預測性——模擬中的世界永遠乾淨整齊，真實的實驗室地板上有電線與灰塵。

縮小這個落差有幾種策略：領域隨機化（在模擬中隨機改變摩擦係數、質量、光照，逼迫策略學會通用）、系統辨識（用真機數據校正模擬參數）、以及混合訓練（模擬中預訓練、真機上微調）。這些方法在第 31 章會詳細討論。

表 2-5 模擬器與實體機器人的比較

面向	模擬器	實體機器人
成本	極低（軟體授權或免費）	高（硬體、維修、耗材）
速度	可快於實時數十倍	受物理限制，無法加速

面向	模擬器	實體機器人
可重複性	完美重現	每次條件略有差異
安全性	失敗無代價	失敗可能損壞或傷人
真實性	有 Sim-to-Real 落差	百分之百真實
適合階段	演算法開發、大量訓練、驗證	最終驗證、實務展示、產業應用

2.7.4 正確的使用姿態

模擬與實機不是二選一，而是分工。本書建議的流程是：先在模擬器中驗證演算法邏輯是否正確（省時間），再在實機上驗證是否能對抗真實世界的雜訊（省面子）。最糟糕的兩種做法是：只在模擬中完成專題，畢業展示時才發現真機根本跑不動；或是完全不用模擬，把整個學期耗在硬體除錯上，演算法一行都沒寫完。

2.8 名校課程巡禮：Stanford CS223A 與 Craig 教科書傳統

2.8.1 史丹佛的機器人學傳統

史丹佛大學是現代機器人學的發源地之一。1969 年 Scheinman 在此設計出 Stanford Arm，奠定了電動六自由度手臂的基本架構；而 John J. Craig 的《Introduction to Robotics: Mechanics and Control》正是源自史丹佛早期的機器人課程講義，這本教科書自 1986 年初版以來，成為全世界最廣泛採用的機器人學教材，也是本書數理骨架的主要參考。

史丹佛的機器人學導論課程（長期由 Oussama Khatib 教授主講）建立了一套經典的教學序列：空間描述與轉換 → 正向運動學（DH 參數）→ 逆向運動學 → 雅可比與速度靜力 → 動力學 → 軌跡生成 → 線性控制 → 非線性控制 → 力控制。這個序列的邏輯極其嚴謹：先描述位置，再描述速度，再描述力，最後控制力——每一步都建立在前一步之上。Khatib 教授本人也是操作空間控制（Operational Space Control）理論的提出者，本書第 17 章即介紹此方法。

2.8.2 這個傳統的優點與侷限

Craig 傳統的最大優點是數理嚴謹、體系完整：讀完之後，你能真正計算一台機器手臂該輸出多少力矩。它至今仍是所有機器人課程的骨幹，本書第二篇與第三篇即依循此架構。

但它也有明確的侷限：成書年代較早，內容集中於固定基座的操作臂，對移動機器人、機率狀態估測、SLAM、機器學習與現代 AI 幾乎沒有著墨。因此本書在保留其骨架的同時，補上了第

四篇與第五篇——這正是「經典骨架 + 現代血肉」的編寫策略。

2.9 MIT：機器人導論與現代機器操作

2.9.1 系統整合的教學傳統

麻省理工學院的機器人課程以「系統整合」見長。其機器人導論課程涵蓋平面與空間運動學、運動規劃、操作臂與移動機器人的機構設計、多剛體動力學、控制器、致動器、感測器、嵌入式軟體，並以動手專題收尾。與史丹佛的「深挖操作臂數學」相比，MIT 更強調「一台完整的機器人系統需要哪些零件，以及它們如何協同」。

這種取向對台灣的工程教育特別有參考價值：許多學生數學考得很好，卻無法把馬達、感測器、控制器與程式整合成一台會動的機器。本書第一篇（第 3~7 章）用整整五章講機器人的身體與系統，正是採納了這個教學傳統。

2.9.2 現代機器操作：把深度學習接進來

MIT 近年的現代機器操作課程（Robotic Manipulation）代表了另一條前沿路線：它把深度學習與三維幾何感知、無碰撞運動規劃、任務與動作規劃、不確定性下的規劃，以及模型式與學習式控制，整合成一個完整的操作系統。課程的核心問題不再是「如何讓手臂到達某個座標」，而是「如何讓機器人在雜亂的桌面上，看到一個沒見過的物體並把它撿起來」。

這個轉變非常關鍵：它代表機器人學的重心，正從「已知環境中的精確運動」移向「未知環境中的穩健操作」。本書第 21 章（抓取）、第 22 章（視覺）與第 31~33 章（學習與具身智慧）即呼應此方向。

2.10 CMU：感測、思考、身體與互動四大領域

2.10.1 四大支柱的架構

卡內基美隆大學擁有全球規模最大的機器人研究機構之一（Robotics Institute），其課程架構以四個基礎領域組成：感測與感知（Sensing and Perception）、動作與思考（Cognition / Reasoning）、機器人的身體（Mechanisms / Robot Bodies），以及與環境的互動（Interaction with the Environment）。

這個四分法的洞見在於：它不把機器人視為一台機械裝置，而視為一個「感知—推理—規

劃—行動」的完整智慧系統。任何一個機器人問題，都可以被定位到這四個領域之一，或它們的交集。這也是本書採用「感知—思考—行動」作為第 1 章核心框架的原因。

2.10.2 自主移動與 SLAM 的重鎮

CMU 同時是自主移動機器人與 SLAM 研究的重鎮。其 16-311 機器人導論課程涵蓋 PID 控制、Bug 演算法、非完整約束與運動規劃；16-833 機器人定位與建圖課程則專攻機率狀態估測、粒子濾波、圖最佳化 SLAM 等主題。本書第四篇（第 22～28 章）的架構，以及本書系另一本《地圖建構與同步定位》三十一章教科書，皆大量參考此教學脈絡。

◎ 給學生的觀察

比較這三所學校，你會看到一個有趣的分工：史丹佛把「操作臂的數學」做到極致，MIT 把「系統整合與現代操作」做到極致，CMU 把「感知與自主移動」做到極致。沒有一所學校能同時做完全部——這也提醒我們，機器人學是一門必須靠團隊分工的學科。你不必全能，但必須知道自已的位置，以及如何與其他位置的人合作。

2.11 柏林工業大學與 ETH Zürich：圖解教學與 ROS 2 生態

2.11.1 德式教學：大量圖解與系統結構

德國的機器人教材有一個鮮明的特色：圖片極多、系統結構清晰、循序解說。本書的重要參考之一，是柏林工業大學 Marc Toussaint 教授的機器人學講義，其內容涵蓋運動學、動力學、路徑規劃（Bug、勢場、PRM、RRT）、機率、移動機器人、SLAM、控制理論（HJB、Riccati、Lyapunov）、強化學習、抓取與腿式運動——範圍遠比傳統操作臂教材寬廣，且大量使用示意圖降低理解門檻。

這種教學風格對初學者極為友善：一張好的示意圖，往往勝過三頁的文字推導。本書在每個重要概念都盡量安排「一張系統架構圖、一張座標示意圖」，即是採納此精神。同時，Toussaint 講義中對最佳控制、可控性與 Lyapunov 穩定性的完整處理，也是本書第 18、19 章獨立成章的依據。

2.11.2 ETH Zürich：ROS 2 與機器人學習

蘇黎世聯邦理工學院代表了機器人教育的另一個前沿。其 ROS 2 課程系統性地涵蓋節點、主題、訊息、服務、動作、URDF 模型、Gazebo 模擬、RViz 視覺化、TF 座標轉換，以及感測器

與致動器的閉迴路控制整合——這是今日機器人軟體工程的實務標準，本書第 29 章即依此架構編寫。

其機器人學習課程則更進一步，涵蓋模仿學習、強化學習、策略學習、生成模型、Transformer、視覺—語言—動作（VLA）模型與機器人基礎模型。ETH 同時也是四足機器人 ANYmal 的誕生地，在腿式運動與強化學習控制方面有標誌性的成果——本書第 28 章與第 31 章的內容方向即與此呼應。

表 2-6 五所名校機器人課程的側重比較

學校	代表課程 / 教材	核心側重	本書對應章節
Stanford	CS223A、Craig 教科書	操作臂數學：運動學、動力學、控制	第二、三篇（第 8~21 章）
MIT	機器人導論、現代機器操作	系統整合、感知與學習式操作	第一篇、第 21~22、31 章
CMU	16-311、16-833	感知、自主移動、SLAM	第四篇（第 22~28 章）
TU Berlin	Toussaint 機器人講義	圖解教學、規劃、最佳控制、機率	第 18~19、24~26 章
ETH Zürich	ROS 2、機器人學習	軟體工程、腿式運動、具身 AI	第 28~33 章

2.11.3 台灣的位置與本書的取舍

巡禮一圈之後，回到台灣。我們的優勢在於：精密機械與電子產業聚落完整，學生有大量接觸真實設備的機會；技職教育體系動手能力強。我們的挑戰在於：數理課程與實作課程常常各自為政，學生要嘛只會算不會做，要嘛只會做不會算。

因此本書的取舍是：以 Craig 的數理骨架保證嚴謹（不能只會做）、以德式圖解降低門檻（不能被公式擋住）、以 MIT 與 CMU 的系統與感知架構補足現代性（不能只有操作臂）、以 ETH 的 ROS 2 與具身 AI 銜接前沿（不能與國際脫節）、再以台灣的產業案例與社團實作落地（不能與在地脫節）。這五條線交織起來，就是你手上這本書。

2.12 問題導向學習與專題導向學習

2.12.1 兩種學習法的差異

問題導向學習 (Problem-Based Learning, PBL) 以一個真實問題為起點：「如何讓機器人在停電後知道自己在哪裡？」學習者為了解決問題，被迫去學習所需的知識。問題本身沒有標準答案，學習的軌跡也因人而異。

專題導向學習 (Project-Based Learning) 則以一件具體作品為終點：「做出一台能自主巡邏教室並回報異常的機器人。」它有明確的交付物、時程與規格，過程中需要分工、整合與測試。

兩者常被混用，但差別很重要：PBL 的核心是「理解」，專題的核心是「產出」。機器人學的理想教學是兩者結合——用問題驅動理解，用專題驗證能力。本書每章的「課堂討論」偏向 PBL，「動手做與專題延伸」偏向專題導向，第 36 章則是完整的總整專題方法論。

表 2-7 問題導向與專題導向學習的比較

面向	問題導向學習 (PBL)	專題導向學習
起點	一個真實問題	一個作品目標
核心	理解與探究	產出與整合
成果	解法與洞見	可展示的作品
評量	推理過程、學習歷程	作品完成度、規格達成率
時程	彈性，可短可長	明確，有里程碑
本書對應	課堂討論、叫獸開講	動手做、附錄實作、第 36 章

2.12.2 設計一個好的入門專題

一個好的機器人入門專題應該符合五個條件。第一，可完成：以現有的時間、預算與能力，確實做得出來——過度野心的專題百分之九十以死機收場。第二，可測量：有客觀的成功標準，例如「循跡完成一圈的時間低於三十秒」，而非「跑得很順」。第三，可分解：能拆成機構、電子、程式、整合四個子任務，讓團隊分工。第四，有痛點：在製作過程中必定會遇到某個技術瓶頸——這個瓶頸正是學習發生的地方。第五，可延伸：完成後能加上新功能，成為下一圈螺旋的起點。

以循跡小車為例：可完成（一週）、可測量（單圈秒數與出界次數）、可分解（車架 / 馬達驅動 / 感測器 / 程式）、有痛點（轉彎衝出去→需要 PID）、可延伸（加超音波避障→加相機辨識號誌）。這正是它成為全世界機器人入門經典的原因。

2.12.3 從失敗中學習

機器人專題最重要的教育價值，往往來自失敗。馬達燒掉、程式當機、比賽當天車子不動——這些挫折會逼出真正的工程能力：系統性除錯、風險備援、時間管理。本書第 36 章特別設有「失敗原因分析與設計改良」一節，並建議所有專題都要撰寫「失敗紀錄」：記下每一次故障的現象、猜測、驗證與結論。三年後你會發現，這份紀錄比任何一份成功報告都更有價值。

2.13 三種讀法與學習環境建置

2.13.1 大學生讀法

大學修課讀法建議依章節順序閱讀，並在每章完成三件事：讀懂例題、跑通程式、完成動手做。時程上，第一學期可完成導讀篇與第一、二篇（第 1~12 章），重點在建立機構直覺與座標運動學基礎；第二學期進入第三、四篇（第 13~28 章），重點在控制與感知；第三學期（或大三上）進入第五、六篇（第 29~36 章）與總整專題。若課程只有一學期，建議路徑為：第 1~8 章 → 第 10~11 章 → 第 15~16 章 → 第 22~26 章 → 第 29 章，捨去最佳控制與具身 AI 的深入部分。

2.13.2 高中專題讀法

高中生以故事層與圖解為主，跳過所有「進階推導」與矩陣推導細節。建議路徑為：第 1~7 章（完整讀，建立全貌與硬體知識）→ 第 16 章 PID（只讀直覺與實作，跳過微分方程解）→ 第 22 章視覺（重點在 YOLO 應用）→ 第 32 章 LLM 應用 → 附錄 F~L 的實作單元。搭配科展或專題競賽，一年內足以完成一件有深度的作品。

2.13.3 國中小社團讀法

完全從附錄 A 開始，動手優先。每完成一台機器人，翻回「螺旋銜接」欄指定的章節，只讀該章的「國中小也看得懂」專欄。建議順序為：附錄 A 震動機器人 → B 投石戰車 → F 循跡小車 → G 超音波避障 → H 藍牙遙控 → D 四伺服雙足 → J 機器狗 → K YOLO 資源回收 → L VLM 機器人。這條路線橫跨機構、控制、感知與 AI，等於用九台機器人走完一遍機器人學的全景。

圖 2-2 三種讀法的路徑圖

建議配圖：以本書 36 章與 12 附錄為橫軸方格，用三條不同顏色的路徑線標出大學生（依序全走）、高中生（跳躍式）、國中小（由附錄起跳並回連正文）的行進路線。

2.13.4 軟硬體環境建置

本書的實作分為三個層級，讀者可依資源選擇。輕量級（預算約一千至三千元）：micro:bit 或 Arduino Uno、直流馬達與驅動模組、紅外線循跡感測器、超音波模組、伺服馬達數顆、麵包板與杜邦線。中量級（約五千至一萬五千元）：ESP32 開發板、無刷馬達與電變、IMU 模組、藍牙 / WiFi 模組、六軸小型機械手臂套件、鋰電池與充電器。重量級（實驗室等級）：Raspberry Pi 5 或 NVIDIA Jetson、RGB-D 相機、二維光達、差速輪底盤、協作機械手臂（如有）。

軟體方面則建議建置以下環境：Python 3 與 NumPy、Matplotlib、OpenCV；Arduino IDE 或 PlatformIO；Ubuntu 系統與 ROS 2（可用虛擬機或 WSL2）；Gazebo 或 Webots 模擬器；Git 版本控制；以及一套順手的編輯器（VS Code 為主流選擇）。第 29 章會提供完整的安裝步驟。

表 2-8 三個層級的軟硬體建置清單

層級	主要硬體	主要軟體	適用對象	可完成的本書內容
輕量級	micro:bit / Arduino、直流馬達、紅外線與超音波感測器、伺服馬達	圖形化程式、Arduino IDE	國中小社團、高中入門	附錄 A~H、第 16 章 PID 實作
中量級	ESP32、IMU、無刷馬達、小型機械手臂套件、鋰電池	Python、Arduino C++、簡易模擬器	高中專題、大學一二年級	附錄 D~J、第 8~17 章實作
重量級	Raspberry Pi 5 / Jetson、RGB-D 相機、光達、差速底盤	Ubuntu、ROS 2、Gazebo、PyTorch	大學專業課程、專題研究	全書，含第 22~33 章

完整計算例題

例題 2-1 學習時數的規劃

題目：某大學生每週可投入機器人學習 12 小時，一學期以 18 週計。若希望在一學期內完成本書第 1~12 章（共 12 章），且每章需要「閱讀 2 小時、例題與程式 2 小時、動手做 3 小時」，請評估此規劃是否可行，並提出調整建議。

解：每章所需時數 = $2 + 2 + 3 = 7$ (小時)。12 章共需 $12 \times 7 = 84$ (小時)。一學期可投入時數 = $12 \times 18 = 216$ (小時)。表面上時數充裕 ($84 \div 216 \approx 39\%$)。但實務上必須保留：數學補救 (約 30 小時)、硬體採購與除錯 (約 40 小時，機器人專案的除錯時間通常等於或大於開發時間)、以及期中期末考週的中斷 (約 4 週、48 小時)。調整後可用時數 $\approx 216 - 30 - 40 - 48 = 98$ (小時)，仍高於 84 小時，規劃可行但緩衝不大。建議：將第 9~12 章的動手做部分改為模擬器實作 (省去硬體除錯時間)，或把第 12 章移至下學期。

例題 2-2 專題可行性評估

題目：三人團隊有 10 週時間、預算 8000 元，欲製作「能在教室內自主巡邏並辨識障礙物的機器人」。已知：底盤與馬達 2500 元、Raspberry Pi 與相機 4000 元、感測器與電池 1500 元。請評估可行性並拆解工作。

解：預算 = $2500 + 4000 + 1500 = 8000$ (元)，恰好用盡，無備品緩衝——建議先削減至 7000 元以保留 1000 元的損壞備援 (機器人專案中馬達與感測器的損耗率不可忽視)。時程拆解建議：第 1~2 週機構組裝與馬達驅動測試；第 3~4 週感測器整合與資料讀取；第 5~6 週自主移動與避障邏輯；第 7~8 週影像辨識整合；第 9 週系統整合測試；第 10 週報告與展示。人力分工：一人負責機構與電子、一人負責移動控制、一人負責影像辨識，整合階段全員投入。風險點：影像辨識 (第 7~8 週) 若延誤將壓縮整合時間，建議提前於第 5 週開始平行進行資料蒐集與模型訓練。

程式範例：檢查你的開發環境

以下 Python 程式檢查本書所需的套件是否安裝完成，並印出版本資訊。請在開始第 3 章之前執行一次，確認環境無誤。

```
import sys

print(f"Python 版本: {sys.version.split()[0]}")
print("-" * 40)

packages = {
    "numpy":      "矩陣與向量運算 (第 8 章起大量使用)",
    "matplotlib": "繪圖與資料視覺化",
    "scipy":      "科學計算、最佳化與訊號處理",
    "cv2":        "OpenCV 影像處理 (第 22 章)",
}
```



```

missing = []
for name, purpose in packages.items():
    try:
        mod = __import__(name)
        ver = getattr(mod, "__version__", "未知")
        print(f"[OK] {name:12s} v{ver:10s} {purpose}")
    except ImportError:
        print(f"[缺少] {name:12s} {'':11s} {purpose}")
        missing.append(name)

print("-" * 40)
if missing:
    pipname = {"cv2": "opencv-python"}
    cmds = " ".join(pipname.get(m, m) for m in missing)
    print(f"請執行安裝指令：pip install {cmds}")
else:
    print("環境檢查完成，可以開始第 3 章了！")

```

若出現缺少套件的訊息，依提示執行 `pip` 安裝即可。建議使用虛擬環境（`venv` 或 `conda`）管理本書專用的套件，避免與其他專案衝突。

ROS 2 與模擬實作：建立你的學習工作區

本章的實作目標是建立一個往後三十四章都會用到的工作區與筆記系統。步驟如下：

1. 在電腦上建立資料夾結構：`robotics_course/`，其下分為 `notes/`（筆記）、`code/`（各程式）、`sim/`（模擬檔案）、`hardware/`（電路圖與零件清單）、`logs/`（實驗與失敗紀錄）。
2. 在該資料夾執行 `git init`，建立版本控制。撰寫第一份 `README.md`，寫下你的學習目標與預定進度。
3. 安裝 Python 環境並執行上一節的環境檢查程式，將輸出結果貼入 `notes/ch02_env.md`。
4. （進階）安裝 Ubuntu 與 ROS 2，或在 Windows 上啟用 WSL2 安裝。第 29 章會用到，提前安裝可避免屆時卡關。
5. 建立 `logs/failures.md`，格式為：日期 | 現象 | 猜測原因 | 驗證方法 | 結論。往後每一次除錯都記錄一則。

這五個步驟看似瑣碎，卻是專業與業餘的分水嶺。三十四章之後，這個資料夾會成為你最有價值的作品集——包括那份失敗紀錄。

國中小也看得懂：學機器人就像學騎腳踏車

學做機器人，就像學騎腳踏車。你不會先讀完一本《腳踏車力學原理》才敢上車，對不對？你會先跨上去、踩踩看、摔幾次、慢慢就會了。等你騎得很順之後，如果想騎得更快、想爬更陡的山，這時候你才會想知道：齒輪怎麼調？坐墊多高最省力？這些「原理」就變得有用了。學機器人也一樣：先做一台會動的小車（附錄 A 到 F），玩到你開始問「為什麼它轉彎會衝出去？」，那就是你該翻開課本的時候了。記住兩件事：第一，先動手，再問為什麼；第二，壞掉不要怕，每個工程師的桌上都有一堆燒掉的零件，那是他們學會的證據。

叫獸提醒與常見錯誤

- 誤區一：等數學全部學完才開始。數學是工具不是門票，用到再學（即時學習）效率最高，也記得最牢。
- 誤區二：只抄程式碼不改參數。抄來的程式能跑，不代表你懂；把 K_p 改成兩倍看看會怎樣，才是真的在學。
- 誤區三：跳過模擬直接上實機。實機除錯耗時是模擬的十倍以上，先在模擬中確認邏輯正確，再上機驗證物理。
- 誤區四：迷信模擬結果。模擬中完美的抓取，在真機上常常滑落。Sim-to-Real 落差是必然的，必須實測。
- 誤區五：不做版本控制。「昨天明明可以跑」是機器人實驗室最常聽到的哀嚎，Git 是最便宜的保險。
- 誤區六：專題訂得太大。十週的專題請訂五週能做完的目標，剩下五週會被除錯與整合吃掉——這是經驗法則，不是悲觀。
- 誤區七：失敗不記錄。同一個錯誤犯第二次，代價是雙倍的時間。失敗紀錄是機器人工程師最重要的個人資產。

課堂討論

1. 阿哲與小玆的爭論，你比較同意誰？請提出第三種路線，並說明它如何避開兩人各自的缺點。
2. 以你目前的數學程度，本書哪一章最可能讓你卡住？你打算用什麼策略處理（先跳過、即時

補、還是提前準備)？

3. 如果系上只能開一門機器人課(一學期三學分)，你認為應該教哪些主題？請從本書 36 章中選出十章並說明理由。
4. 模擬器讓學習成本大幅下降，但也可能讓學生失去接觸真實硬體的機會。你認為理想的模擬與實機比例是多少？為什麼？
5. 五所名校的課程各有側重，如果要為台灣的產業需求設計一門課，你會偏向哪一所的風格？請結合第 1 章的台灣產業內容作答。

章末摘要

- 機器人學的知識可分為六層金字塔：物理數學、機構硬體、幾何運動學、力學控制、感知規劃、學習智慧；層間依賴分硬性與軟性，硬性補足、軟性可跳。
- 最有效的學習法是螺旋式而非線性：觀察 → 組裝 → 程式 → 數學模型，每繞一圈加深一層，同一主題反覆出現於不同深度。
- 四大數學工具各司其職：線性代數管空間、微積分管變化、機率論管不確定性、最佳化管選擇；深度以扎實的大一大二程度為足。
- 程式工具三分工：圖形化入門、Python 做原型與演算法、C/C++ 做即時與嵌入式；版本控制、註解與模組化是三樣必備工程習慣。
- 模擬器提供零成本、高速、可重複與安全的試錯環境，但存在 Sim-to-Real 落差；正確姿態是先模擬驗證邏輯、再實機驗證物理。
- 五所名校側重不同：Stanford 深耕操作臂數學、MIT 強調系統整合與現代操作、CMU 專精感知與 SLAM、TU Berlin 圖解與規劃控制、ETH 領先 ROS 2 與具身 AI。
- 本書策略為五線交織：Craig 數理骨架、德式圖解、MIT 與 CMU 的系統與感知、ETH 的軟體與前沿、台灣產業與社團實作。
- 好的入門專題需符合五條件：可完成、可測量、可分解、有痛點、可延伸；失敗紀錄的價值往往高於成功報告。
- 三種讀法各有路徑：大學生依序精讀、高中生跳躍取用、國中小由附錄起跳並回連正文專欄。

觀念題與計算題

觀念題

1. 畫出機器人學的六層知識階層圖，並各舉一個屬於該層的具體主題。
2. 說明「硬性依賴」與「軟性依賴」的差別，並各舉一例。
3. 以自己的話解釋螺旋式學習，並以「讓車子穩定行走」為主題設計三圈學習循環。
4. 線性代數、微積分、機率論、最佳化四大工具，各負責機器人學中的什麼問題？各舉一個本書章節。
5. 為什麼程式中計算角度應使用 $\text{atan2}(y, x)$ 而非 $\text{atan}(y/x)$ ？
6. 旋轉矩陣是正交矩陣，這個性質在工程上帶來什麼實質好處？
7. 圖形化程式、Python 與 C/C++ 在機器人開發中的分工為何？各舉一個適用場合。
8. 列舉模擬器的四個價值，以及 Sim-to-Real 落差的四個來源。
9. 比較問題導向學習與專題導向學習的差異，並說明本書哪些欄位分別對應此二者。
10. 說明一個好的入門專題應符合的五個條件，並用循跡小車逐條驗證。
11. 比較 Stanford、MIT、CMU 三校機器人課程的側重差異，並說明本書如何整合三者。
12. 為什麼「失敗紀錄」對機器人工程師特別重要？請從機器人學的學科特性說明。

計算題

1. 某學生每週可投入 8 小時學習，一學期 18 週。若每章需 7 小時，且需保留 25 小時數學補救、35 小時硬體除錯、兩週考試中斷，請計算他一學期最多能完成幾章，並建議應選讀本書哪幾章。
2. 四人團隊有 12 週、預算 15000 元，欲製作一台具 SLAM 建圖功能的巡邏機器人。已知底盤 4000 元、Jetson 6000 元、二維光達 4000 元、電池與雜項 1500 元。請計算預算餘額、評估是否需要調整，並提出十二週的里程碑分工表（至少四個里程碑）。

動手做與專題延伸

本章實作：擬定你的個人學習計畫

完成下表，作為往後一年的學習契約。建議貼在書桌前，每月檢視一次進度。

表 2-9 個人學習計畫表 (範例列供參考)

項目	內容	自我評估 / 填寫
我的身分	大學生 / 高中生 / 社團學生 / 自學者	(填寫)
每週可投入時數	含閱讀、程式與動手做	(填寫)
目標讀完章節	例：一學期完成第 1 ~ 12 章	(填寫)
數學缺口	例：機率論尚未修過，第 24 章前需補	(填寫)
硬體層級	輕量級 / 中量級 / 重量級	(填寫)
第一圈螺旋專題	例：循跡小車，四週完成	(填寫)
第二圈螺旋專題	例：自平衡兩輪車，八週完成	(填寫)
檢核里程碑	例：第 8 週前跑通第一支 Python 控制程式	(填寫)

專題延伸

- 上網查閱本章提到的五所名校中任一所的機器人課程大綱 (多數公開於課程網站)，與本書目次逐章比對，列出三個本書涵蓋而該課程未涵蓋的主題，以及三個相反的情形。
- 訪問一位大三以上的學長姐或研究生，請教他們「如果重來一次，會怎麼安排機器人學的學習順序」，並將建議整理成五百字，與本章的建議比較異同。
- 進階：挑選一個你感興趣的機器人主題 (例如四足步態或視覺 SLAM)，用本章的四階段學習法，規劃一份為期十二週的自學計畫，明確列出每階段的產出與驗收標準。

參考文獻與延伸閱讀

- Craig, J. J., Introduction to Robotics: Mechanics and Control, 4th ed., Pearson.
- Siciliano, B., Sciavicco, L., Villani, L. and Oriolo, G., Robotics: Modelling, Planning and Control, Springer.
- Lynch, K. M. and Park, F. C., Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press.
- Toussaint, M., Introduction to Robotics 課程講義, TU Berlin / University of Stuttgart.
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press.
- LaValle, S. M., Planning Algorithms, Cambridge University Press (線上全文公開)。
- Strang, G., Introduction to Linear Algebra, Wellesley-Cambridge Press (線性代數推薦教

材)。

- Macnab, A., "Spiral Curriculum" 相關教育學文獻；Bruner, J. S., The Process of Education, Harvard University Press (螺旋式課程的理論來源)。
- ROS 2 官方文件 (docs.ros.org) 與 Gazebo、Webots 官方教學資源。
- 各校公開課程資源：Stanford CS223A、MIT OpenCourseWare 機器人相關課程、CMU 16-311 / 16-833 課程網站、ETH Zürich ROS 2 課程。